

[Dashboard](#) / [My courses](#) / [ITB_IF2010_2_2526](#) / [Praktikum 5](#) / [Post Praktikum 5](#)

Started on	Tuesday, 12 May 2026, 7:30 PM
State	Finished
Completed on	Tuesday, 12 May 2026, 8:09 PM
Time taken	39 mins 10 secs
Grade	400.00 out of 400.00 (100%)

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Kebin sedang mendalami konsep **Interface** dalam *Object Oriented Programming*. Setelah memahami teori tentang abstraksi dan polymorphism, ia mencoba mempraktikkannya dengan membuat sebuah sistem pembayaran sederhana menggunakan konsep Payment Strategy. Dalam proses belajarnya, Kebin ingin merancang sistem *checkout* yang fleksibel sehingga metode pembayaran dapat diganti kapan saja tanpa perlu mengubah logika utama program.

Bantulah Kebin mengimplementasikan fitur pembayaran ini dengan melengkapi kelas-kelas dan antarmuka (*interface*) berikut:

- **PaymentStrategy** (Interface): Memiliki metode `void pay(int amount)`.
- **CardPayment**: Implementasi **PaymentStrategy**. Output `pay` adalah `Paid $<amount> using Credit Card`.
- **BankTransferPayment**: Implementasi **PaymentStrategy**. Output `pay` adalah `Paid $<amount> using Bank Transfer`.
- **EWalletPayment**: Implementasi **PaymentStrategy**. Output `pay` adalah `Paid $<amount> using E-Wallet`.
- **Checkout**: Memiliki atribut `paymentStrategy` (tipe **PaymentStrategy**), dengan metode `void setPaymentStrategy(PaymentStrategy)` dan `void processPayment(int amount)` yang memanggil metode `pay`. Jika `paymentStrategy` masih `null` saat `processPayment` dipanggil, output adalah: `No payment method selected`.

Driver tersedia pada [Main.java](#)

Input:

Baris pertama berisi **N** yang merepresentasikan jumlah perintah (N baris setelahnya).

Perintah dapat berupa:

- `SET CARD`
- `SET EWALLET`
- `SET BANK`
- `PAY <amount>`

Contoh Input:

- `2`
- `SET CARD`
- `PAY 100`

Contoh Output:

`Paid $100 using Credit Card`

Pengumpulan:

Kumpulkan Checkout.java, PaymentStrategy.java, BankTransferPayment.java, EWalletPayment.java, CardPayment.java pada **Checkout.zip**

Setiap output harus menggunakan newline

Java 8 ▾

 [Checkout.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.11 sec, 28.84 MB
2	20	Accepted	0.09 sec, 28.79 MB

No	Score	Verdict	Description
3	20	Accepted	0.11 sec, 28.61 MB
4	20	Accepted	0.08 sec, 28.92 MB
5	20	Accepted	0.11 sec, 28.05 MB

Question **2**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Gro sedang membangun sistem penyimpanan data untuk gudang risetnya. Gudangnya memiliki tiga laci khusus: satu untuk bilangan bulat (*integer*), satu untuk bilangan desimal (*double*), dan satu untuk kata-kata (*string*).

Yang membuatnya menarik: semua laci ini berbagi *struktur yang sama*, hanya tipe datanya yang berbeda. Dinda ingin menggunakan **Java Generics** agar kode tidak perlu ditulis ulang untuk setiap tipe.

File **Main.java** sudah disediakan dan **tidak boleh diubah**. Tugasmu adalah mengimplementasikan kelas-kelas generik yang digunakan oleh **Main.java**.

File yang Disediakan (tidak boleh diubah):

- [Main.java](#)

File yang Harus Dibuat:

- Laci.java
- LaciAngka.java
- LaciUtil.java

Spesifikasi Kelas

1. Kelas Generik Laci<T>

Kelas generik yang menyimpan item bertipe **T** dalam array dengan kapasitas tetap **10**. Implementasikan method-method berikut:

- **Laci(String label)** — konstruktor, simpan label laci
- **boolean simpan(T item)** — tambah item ke laci. Kembalikan **true** jika berhasil, **false** jika laci sudah penuh
- **T ambil(int i)** — kembalikan item pada indeks ke-**i** (dimulai dari 1). Kembalikan **null** jika indeks tidak valid
- **void set(int i, T item)** — ganti item pada indeks ke-**i** (dimulai dari 1). Abaikan jika indeks tidak valid
- **int ukuran()** — kembalikan jumlah item saat ini
- **String getLabel()** — kembalikan label laci
- **String toString()** — format: **Laci[label]: [item1, item2, ...]**

2. Kelas Generik LaciAngka<T extends Number>

Kelas turunan dari **Laci<T>** yang khusus untuk tipe angka (**Integer**, **Double**, dsb.). Gunakan bounded type parameter **T extends Number** agar bisa mengakses method **doubleValue()**. Implementasikan:

- **LaciAngka(String label)** — konstruktor
- **double total()** — jumlahkan semua item, kembalikan sebagai **double**
- **double rataRata()** — kembalikan rata-rata semua item sebagai **double**. Kembalikan **0.0** jika laci kosong

3. Kelas Utilitas LaciUtil

Kelas dengan method **static generik**. Perhatikan tipe parameter pada setiap method:

- **static <T> void tukar(Laci<T> laci, int i, int j)** — tukar posisi item pada indeks ke-**i** dan ke-**j** (1-based).
Tidak perlu tahu tipe spesifik T, cukup pakai tipe parameter generik.
- **static <T extends Comparable<T>> T terbesar(Laci<T> laci)** kembalikan item terbesar dalam laci. Kembalikan **null** jika laci kosong.
Bounded type parameter memastikan item bisa dibandingkan satu sama lain.

Format Output

Perintah	Kondisi	Output
SIMPAN INT <val>	—	Disimpan: val
SIMPAN DBL <val>	—	Disimpan: val
SIMPAN STR <val>	—	Disimpan: val
AMBIL INT/DBL/STR <i>	indeks valid	nilai item ke-i
AMBIL INT/DBL/STR <i>	indeks tidak valid	null
TOTAL INT/DBL	—	Total: X.X
RATA INT/DBL	—	Rata-rata: X.X
TUKAR INT/STR <i> <j>	—	(tidak ada output)
TERBESAR INT/STR	laci tidak kosong	nilai item terbesar
TERBESAR INT/STR	laci kosong	Laci kosong
INFO INT/DBL/STR	—	Laci[label]: [item1, item2, ...]

Contoh Masukan dan Keluaran

Masukan Keluaran

[Masukan](#) [Keluaran](#)

Java 8 ▾

 [answere.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	14	Accepted	0.07 sec, 26.59 MB
2	14	Accepted	0.10 sec, 27.98 MB
3	14	Accepted	0.08 sec, 28.51 MB
4	14	Accepted	0.07 sec, 28.47 MB
5	14	Accepted	0.08 sec, 28.04 MB
6	14	Accepted	0.08 sec, 28.93 MB
7	16	Accepted	0.07 sec, 28.64 MB

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	3 s
Memory limit	64 MB

Jus Buah

Setelah Kebin dan Stewart puas bermain monopoli dan moba, mereka tak terasa mengalami dehidrasi tinggi. Tidak lama kemudian diam-diam mereka berdua mengamati Dr. Neroifa saat sedang tidak bertugas, malah membuka kedai jus. Mereka berdua langsung bergegas menghampiri Dr. Neroifa untuk membeli jus. Setelah mereka membeli jus, Kebin dan Stewart merasa ada yang aneh dengan jus mereka. Didapati bahwa mereka mendapat pesanan jus yang salah. Akhirnya, Dr. Neroifa mengaku bahwa ia masih sering bingung dengan desain mesin pembuat jus tersebut. Anda diminta untuk membantu Dr. Neroifa agar pengelolaan kedai jus dapat berjalan dengan baik pada: **DapurJus.java**. Dr. Neroifa hanya menjual beberapa buah saat ini, antara lain, **Stroberi**, **Pisang**, dan **Apel**. Tolong gunakan salah satu konsep OOP agar Dr. Neroifa tidak kesulitan jika ingin mengembangkan bisnisnya, misalnya ingin menambahkan buah **Melon**, **Ceri**, **Jeruk**, **Kiwi**, **Persik**, dan **Alpukat**. (sebuah referensi ytta)

Spesifikasi Kelas

Nama Kelas	Spesifikasi
DapurJus	- DapurJus merupakan utility class yang menyediakan operasi untuk memproses daftar bahan dan daftar minuman. Kelas ini tidak menyimpan state dan tidak dimaksudkan untuk dibuat sebagai objek. Seluruh method dipanggil langsung melalui nama kelas. - Atribut: tidak ada. - Konstruktor: Kelas ini tidak boleh dapat diinstansiasi dari luar kelas karena hanya berisi operasi statis. - Method: - static void cekBahan(??? daftarBahan) mencetak deskripsi setiap bahan pada daftarBahan. - static int hitungTotalManis(??? daftarBahan) mengembalikan jumlah total tingkat manis dari seluruh bahan pada daftarBahan. - static void buatJusApelDefault(??? daftarMinuman) oleh karena Dr. Neroifa memiliki kebun apel yang selalu panen, ia memiliki standar menu yang selalu ada dalam daftar jus apel sehingga tambahkan secara default dua objek Jus Apel berikut, "Jus Apel Original" dan "Jus Apel Madu" ke dalam daftarMinuman. - static void cetakRakUmum(??? rak) mencetak seluruh isi rak. Method ini dapat menerima list dengan tipe apa pun.

Untuk memudahkan pengujian class, berikut program [Main](#) yang dapat Anda coba beserta file-file lainnya yang terkait

Format Pengumpulan

Kumpulkan file: **DapurJus.java**.

Pastikan program dapat dikompilasi dengan **javac *.java** dan dijalankan dengan **java Main**.

Java 8

 [DapurJus.java](#)

Score: 100

Blackbox

Score: 98

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	4	Accepted	0.22 sec, 31.16 MB

No	Score	Verdict	Description
2	4	Accepted	0.25 sec, 30.08 MB
3	4	Accepted	0.38 sec, 30.18 MB
4	4	Accepted	0.26 sec, 30.09 MB
5	4	Accepted	0.27 sec, 30.03 MB
6	4	Accepted	0.14 sec, 26.60 MB
7	4	Accepted	0.07 sec, 25.21 MB
8	4	Accepted	0.10 sec, 26.69 MB
9	4	Accepted	0.14 sec, 25.03 MB
10	4	Accepted	0.15 sec, 26.67 MB
11	4	Accepted	0.14 sec, 25.00 MB
12	4	Accepted	0.11 sec, 25.00 MB
13	4	Accepted	0.09 sec, 25.16 MB
14	4	Accepted	0.12 sec, 25.01 MB
15	4	Accepted	0.29 sec, 25.00 MB
16	4	Accepted	0.37 sec, 29.99 MB
17	4	Accepted	0.43 sec, 30.10 MB
18	4	Accepted	0.38 sec, 30.06 MB
19	4	Accepted	0.28 sec, 30.17 MB
20	4	Accepted	0.45 sec, 30.01 MB
21	4	Accepted	0.10 sec, 26.60 MB
22	4	Accepted	0.18 sec, 26.68 MB
23	4	Accepted	0.31 sec, 25.10 MB
24	2	Accepted	0.25 sec, 24.99 MB
25	4	Accepted	0.20 sec, 25.02 MB

Whitebox

Score: 2

No	Score	Checker	Description
1	2	PMD	

Question **4**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Board Game Analytics

Kumpulkan File: BoardGameAnalytics.java (TANPA fungsi main)

Pada soal ini, Anda diminta melengkapi implementasi kelas `BoardGameAnalytics` untuk mengelola data board game, rating pemain, dan stok game.

File yang Disediakan

- Kelas model `BoardGame`.
- Starter file `BoardGameAnalytics`.

Atribut pada BoardGameAnalytics

- `List<BoardGame> games` untuk menyimpan semua board game.
- `Set<String> players` untuk menyimpan nama pemain unik tanpa duplikasi.
- `Map<String, Integer> stockByGame` untuk menyimpan stok game berdasarkan nama game.
- `Map<String, List<Integer>> ratings` untuk menyimpan semua rating untuk tiap game.

Tugas

Lengkapi method berikut:

- `public BoardGameAnalytics()`
- `public void addGame(BoardGame game, int initialStock)`
- `public void addRating(String gameName, String playerName, int rating)`
- `private double getAverageRating(String gameName)`
- `public List<String> getLowStockGames(int threshold)`
- `public List<String> getRecommendedGames(int playerCount, double minRating)`

Spesifikasi Method

`BoardGameAnalytics()`
Menginisialisasi semua collection yang dibutuhkan.

`addGame(BoardGame game, int initialStock)`
Menambahkan game baru beserta stok awal. Jika sudah ada game dengan nama yang sama, game tidak ditambahkan lagi ke `games`, tetapi stok game tersebut tetap ditambahkan ke `stockByGame`.

`addRating(String gameName, String playerName, int rating)`
Menambahkan rating untuk game bernama `gameName`. Nama pemain harus disimpan di `players` agar setiap pemain hanya tercatat satu kali.

`getAverageRating(String gameName)`
Mengembalikan rata-rata rating game tersebut. Jika belum ada rating, kembalikan `0.0`.

`getLowStockGames(int threshold)`
Mengembalikan daftar nama game yang memiliki stok kurang dari `threshold`. Hasil harus diurutkan berdasarkan stok menaik. Jika terdapat beberapa game dengan stok sama, urutkan berdasarkan nama game secara alfabetis.

`getRecommendedGames(int playerCount, double minRating)`
Mengembalikan daftar nama game yang memenuhi seluruh syarat berikut:

- Game dapat dimainkan oleh `playerCount` pemain, yaitu `minPlayers <= playerCount <= maxPlayers`.
- Game memiliki rata-rata rating minimal `minRating`.

Hasil harus diurutkan secara alfabetis menaik.

Ketentuan

- Wajib menggunakan Java Collection Framework untuk menyimpan dan mengelola data.
- Wajib menggunakan Stream API pada `getAverageRating`, `getLowStockGames`, dan `getRecommendedGames`.

Format Input (pada Main yang telah disediakan)

```
N
name minPlayers maxPlayers playTime category stock
... (sebanyak N baris)
R
gameName playerName rating
... (sebanyak R baris)
threshold
playerCount minRating
```


Keterangan:

- **N** adalah jumlah board game.
- Setiap baris game berisi nama game, jumlah pemain minimum, jumlah pemain maksimum, waktu bermain, kategori, dan stok awal.
- **R** adalah jumlah rating.
- Setiap baris rating berisi nama game, nama pemain, dan nilai rating.
- **threshold** digunakan untuk memanggil `getLowStockGames`.
- **playerCount** dan **minRating** digunakan untuk memanggil `getRecommendedGames`.

Format Output

Program mencetak tepat dua baris.

Baris pertama:

LOW_STOCK <name1> <name2> ...

Jika tidak ada game dengan stok di bawah threshold, cetak:

LOW_STOCK -

Baris kedua:

RECOMMENDED <name1> <name2> ...

Jika tidak ada game yang direkomendasikan, cetak:

RECOMMENDED -

Contoh

Input

```
4
Catan 3 4 90 Strategy 5
Dobble 2 8 15 Party 2
Chess 2 2 30 Abstract 1
Pandemic 2 4 45 Cooperative 6
5
Catan alice 9
Catan bob 8
Dobble charlie 7
Dobble alice 6
Pandemic dave 8
3
3 7.0
```

Output

```
LOW_STOCK Chess Dobble
RECOMMENDED Catan Pandemic
```

Penjelasan Contoh

- **Chess** dan **Dobble** memiliki stok di bawah 3, sehingga muncul pada baris **LOW_STOCK**.
- **Catan** dapat dimainkan oleh 3 pemain dan rata-rata ratingnya 8.5.
- **Pandemic** dapat dimainkan oleh 3 pemain dan rata-rata ratingnya 8.0.
- **Dobble** dapat dimainkan oleh 3 pemain, tetapi rata-rata ratingnya 6.5 sehingga tidak direkomendasikan.
- **Chess** tidak dapat dimainkan oleh 3 pemain.

Java 8 ▾

 [BoardGameAnalytics.java](#)

Score: 100

Blackbox

Score: 80

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	16	Accepted	0.21 sec, 31.99 MB
2	16	Accepted	0.20 sec, 31.87 MB
3	16	Accepted	0.18 sec, 31.51 MB
4	16	Accepted	0.19 sec, 31.45 MB
5	16	Accepted	0.19 sec, 31.88 MB

Whitebox

Score: 20

No	Score	Checker	Description
1	20	PMD	

◀ [Praktikum 5](#)

Jump to...

⚡ [Exception, Assertion, Multithreading, Reflection](#) ▶